TAAOH - Primary Estimand

Setup

```
library(furrr)
library(tidyverse)
library(data.table)
library(arrow)
library(rmsb)
library(here)
library(Hmisc)
library(patchwork)
```

Load Data

We use simulated data with constant OR of 0.8. The OR of 0.8 results in about 1.2 more weeks alive and out of hospital over 26 weeks.

```
set.seed(1234)
df_full <- read_parquet(here("2-hosp", "sim-data", "ha_08.parquet")) |>
    select(id = pat_id, tx, y, yprev, ypprev, week, ecog_fstcnt, diagnosis, albumin,
c_reactive_protein, gender, age)

tx_id <- sample(unique(df_full$id[df_full$tx == 1]), 180, replace = FALSE)
soc_id <- sample(unique(df_full$id[df_full$tx == 0]), 90, replace = FALSE)

df <- df_full |>
  filter(id %in% c(tx_id, soc_id))

# we need to change the patient id to be sequential starting from 1
# otherwise the random effects gammas won't match
df <- df |>
  arrange(id, week) |>
  mutate(id = as.integer(factor(id)))
```

Estimand

We use the potential outcomes framework to formally define the estimand. Let Y_{it}^a be the potential health state for individual i at week t under treatment assignment a, where $a = 1$ is invitation to the single-arm trial (SAT) and $a = 0$ is standard of care (SoC). The health state is an ordinal variable where the state "Alive and Out of Hospital" is denoted by $Y_{it}^a = 1$. The analysis time horizon is $t_{max} = 26$ weeks.

We define an individual-level potential outcome, W_i^a, as the total number of weeks that individual i would spend alive and out of the hospital over the 26-week period if they were assigned to treatment a. This is calculated by summing an indicator function over the time horizon:

$$W_i^a = \sum_{t=1}^{26} \mathbb{I}(Y_{it}^a = 1)$$

where $\mathbb{I}(\cdot)$ is the indicator function, which equals 1 if the patient is in state 1 at week t, and 0 otherwise.

Our primary estimand is the Average Treatment Effect (ATE) on this outcome, denoted τ. It is the difference in the expected total time spent alive and out of the hospital, if everyone had been invited to the SAT ($a = 1$) compared to nobody having been invited to the SAT ($a = 0$).

$$\tau = \mathbb{E}[W_i^1] - \mathbb{E}[W_i^0]$$

The estimand, τ, represents the population-average difference in weeks spent alive and out of the hospital over a 26-week period when comparing the SAT intervention to the SoC.

Fit Model

```
dd <- datadist(df)
options(datadist = "dd")
pcontrast_arg <- list(
  c1 = list(tx = 1),    # <-- Numeric 1 for the treatment group
  c2 = list(tx = 0),    # <-- Numeric 0 for the reference group
  contrast = expression(c1 - c2),
  mu = 0,
  sd = 0.541
)

if(TRUE){
  options(mc.cores = 6)
  model <-
      blrm(
              formula = y  ~ tx + rcs(week, 4) + yprev + ypprev + week %ia% yprev +
week %ia% ypprev + ecog_fstcnt + diagnosis + albumin + c_reactive_protein +
cluster(id),
              data = df,
              pcontrast = pcontrast_arg,
              refresh = 5,
              iter = 2000,
              chains = 4,
              seed = 1234,
              loo = FALSE,
              method = "sampling",
              backend = "cmdstan",
              sampling.args = list(adapt_delta = 0.95)
          )
}
```

Functions

Predictions conditional on random effects

```
predict_blrm_manual <- function(fit, newdata, include_re = FALSE) {
  # --- 0) Prerequisite Checks ---
  if (!is.data.frame(newdata)) {
    stop("`newdata` must be a data frame.")
  }
  if (include_re && !"id" %in% colnames(newdata)) {
    stop("`newdata` must contain a 'id' column when include_re=TRUE")
  }
```

```
  N_new <- nrow(newdata)

  # --- 1) Get the design matrix for the new data ---
  X <- rms::predictrms(fit, newdata = newdata, type = 'x')

  # --- 2) Extract posterior draws and model information ---
  draws <- fit$draws
  ndraws <- nrow(draws)
  ns <- fit$non.slopes
  K <- ns + 1

  intercept_draws <- draws[, 1:ns, drop = FALSE]
  beta_draws <- draws[, (ns + 1):ncol(draws), drop = FALSE]

  # --- 3) Calculate the fixed-effect linear predictor (eta_fixed) ---
  eta_fixed <- beta_draws %*% t(X)

  # --- 4) Get the random effect draws (u_draws) if requested ---
  u_draws <- matrix(0, nrow = ndraws, ncol = N_new)

  if (include_re) {
    if (length(unique(newdata$id)) > 1) {
      stop("`newdata` contains multiple unique patient IDs. This version is restricted
to one at a time.")
    }
    if (!requireNamespace("posterior", quietly = TRUE)) {
      stop("The 'posterior' package is required to extract random effect draws.")
    }
    sf <- rmsb::stanGet(fit)
    all_gamma_draws <- posterior::as_draws_matrix(sf$draws(variables = "gamma"))
    patient_ids <- newdata$id
    u_draws <- all_gamma_draws[, patient_ids, drop = FALSE]
  }

  # --- 5) Calculate the total linear predictor (eta) ---
  stopifnot(all(dim(eta_fixed) == dim(u_draws)))
  eta <- eta_fixed + u_draws

  # --- 6) Calculate probabilities for each observation and each draw ---
  p_cat_draws_list <- vector("list", N_new)

  for (i in 1:N_new) {
    eta_i <- eta[, i]
    s_ge <- plogis(sweep(intercept_draws, 1, eta_i, "+"))

    p_cat_draws <- matrix(NA_real_, nrow = ndraws, ncol = K)
    p_cat_draws[, 1] <- 1 - s_ge[, 1]
    if (K > 2) {
      for (kk in 2:(K - 1)) {
        p_cat_draws[, kk] <- s_ge[, kk - 1] - s_ge[, kk]
      }
    }
    p_cat_draws[, K] <- s_ge[, K - 1]
    colnames(p_cat_draws) <- paste0("y=", 1:K)
```

```
    p_cat_draws_list[[i]] <- p_cat_draws
  }

  # --- 7) Reshape draws into a single 3D array ---
  draws_array <- array(
    data = NA_real_,
    dim = c(ndraws, N_new, K)
  )

  for (i in 1:N_new) {
    draws_array[, i, ] <- p_cat_draws_list[[i]]
  }

  dimnames(draws_array) <- list(
    NULL,
    rownames(newdata),
    colnames(p_cat_draws_list[[1]])
  )

  return(draws_array)
}
```

Calculate SOPs

State Occupancy Probabilities for a Second-Order Process

The state occupancy probability (SOP) is a foundational derived quantity from a transition model. For a second-order process, we define the SOP as $P\big(Y_j = k \mid X, y_{-1}, y_0\big)$, which is the probability of being in state k at time j, conditional on a set of baseline covariates X and the two baseline states, y_{-1} (state at time $j = -1$) and y_0 (state at time $j = 0$).

In a first-order model, a single transition matrix is sufficient to describe the evolution of the system from time $j-1$ to j. In the second-order case, however, the probabilities of transitioning from state k at time $j-1$ to state l at time j are themselves conditional on the state at time $j-2$. This dependency requires a set of transition matrices, one for each possible prior state ($Y_{j-2} = h$), rather than a single matrix. The core challenge is that the state of the system at any given time is not described by a single previous outcome, but by the joint probability distribution of the last two outcomes. Therefore, to compute the SOPs, we must recursively calculate the evolution of this joint probability distribution over time.

Let the conditional transition probability at time j be denoted as $T_j(l \mid h, k) = P\big(Y_j = l \mid Y_{j-2} = h, Y_{j-1} = k, X\big)$. These probabilities, which represent the chance of moving to state l given the history of being in state h then state k, are calculated directly from the fitted ordinal transition model for a specific covariate profile X.

The unconditionalization process proceeds as follows:

Initialization: We begin with the known baseline states y_{-1} and y_0. The joint probability of being in states h and k at times $j = -1$ and $j = 0$ respectively, which we denote as $J_0(h, k) = P(Y_{-1} = h, Y_0 = k)$, is initialized as a matrix of zeros with a 1 at the position $(h, k) = (y_{-1}, y_0)$. The initial SOP at time $j = 0$ is therefore known with certainty: $S_0(k) = 1$ for the state $k = y_0$ and 0 otherwise.

Recursive Calculation of Joint Probabilities: For each subsequent time point $j = 1, 2, \ldots, d$, we calculate the new joint probability distribution $J_j(k, l) = P\big(Y_{j-1} = k, Y_j = l\big)$ by marginalizing over the state at time $j-2$, denoted by h. Using the law of total probability, the recursion is defined as:

$$J_j(k,l) = \sum_{h=1}^{K} T_j(l \mid h,k) \times J_{j-1}(h,k)$$

In this step, we use the joint probabilities for times $j-2$ and $j-1$, J_{j-1}, and the conditional transition probabilities for the current time step, T_j, to compute the joint probabilities for the current and previous time steps, J_j.

Calculation of State Occupancy Probabilities: Once the joint probability matrix J_j for time j is obtained, the SOP for being in state l at time j, denoted $S_j(l)$, is calculated by marginalizing out the state at time $j-1$ (denoted by k):

$$S_j(l) = P(Y_j = l) = \sum_{k=1}^{K} J_j(k,l)$$

This $S_j(l)$ is the marginal probability at time j and is the state occupancy probability $P(Y_j = l \mid X, y_{-1}, y_0)$ for the given baseline conditions. This three-step process is repeated for all time points up to the desired horizon, yielding the full set of SOPs for a subject with a given covariate profile and specific baseline history. The code that implements these calculations can be found here.

```
#' Calculate Second-Order State Occupation Probabilities
#'
#' This function calculates the unconditional state occupation probabilities for a
#' second-order Markov model.
#'
#' @param model_fit The fitted model object (rmsb or brms)
#' @param baseline_data A single-row data frame containing the initial states and
#'   all baseline predictor values required by the model.
#' @param col_yprev A string with the column name for the state at t-1 (e.g., "yprev").
#' @param col_ypprev A string with the column name for the state at t-2 (e.g.,
"ypprev").
#' @param col_time A string with the column name that represents time in the model
(e.g., "week").
#' @param times A numeric vector specifying the time points for calculation (e.g.,
`1:26`).
#' @param n_states An integer for the number of levels in the ordinal outcome.
#' @param absorbing_states A numeric vector listing the absorbing states.
#'
#' @return A list containing two arrays:
#'   \item{unconditional}{A 3D array `(draws, states, time)` with the unconditional
state probabilities.}
#' @export

sop_markov_second_order <- function(model_fit,
                                    baseline_data,
                                    col_yprev,
                                    col_ypprev,
                                    col_time,
                                    times,
                                    n_states,
                                    absorbing_states,
                                    include_re = FALSE) {

  # --- Detect model type and get number of draws ---
  if (inherits(model_fit, "brmsfit")) {
```

```
    detected_type <- "brms"
    if (!requireNamespace("marginaleffects", quietly = TRUE) || !
requireNamespace("posterior", quietly = TRUE)) {
      stop("Packages 'marginaleffects' and 'posterior' are required for brms models.")
    }
    n_draws <- posterior::ndraws(model_fit)
  } else if (inherits(model_fit, "blrm")) {
    detected_type <- "rmsb"
    n_draws <- nrow(model_fit$draws)
  } else {
    stop("Unsupported model object class. Please provide a model from 'brms' (class
'brmsfit') or 'rmsb' (class 'blrm').")
  }

  # --- Parameters ---
  M <- n_states
  T_horizon <- max(times)

  # --- Initial Conditions ---
  initial_state_t_minus_1 <- as.numeric(baseline_data[[col_ypprev]])
  initial_state_t_0 <- as.numeric(baseline_data[[col_yprev]])

  # --- Data Structures to Store Results ---
  P_joint_bayes <- array(0, dim = c(n_draws, M, M, T_horizon + 1),
                         dimnames = list(paste0("draw_", 1:n_draws),
                                         paste0("t-1=", 1:M),
                                         paste0("t=", 1:M),
                                         paste0("time=", 0:T_horizon)))

  P_uncond_bayes <- array(0, dim = c(n_draws, M, T_horizon + 1),
                          dimnames = list(paste0("draw_", 1:n_draws),
                                          paste0("State_", 1:M),
                                          paste0("t=", 0:T_horizon)))

  # Set the known probability at t=0
  P_joint_bayes[, initial_state_t_minus_1, initial_state_t_0, 1] <- 1.0
  P_uncond_bayes[, initial_state_t_0, 1] <- 1.0

  # --- Create the static grid for batch predictions ---
  history_grid <- expand.grid(
    h = factor(1:M, levels = 1:M), # X_{t-2}
    j = factor(1:M, levels = 1:M)  # X_{t-1}
  )
  names(history_grid) <- c(col_ypprev, col_yprev)

  # --- Dynamically create the full prediction dataset ---
  # Identify static covariates by excluding time-varying/history columns
  cols_to_exclude <- c(col_yprev, col_ypprev, col_time)
  static_covariate_names <- setdiff(names(baseline_data), cols_to_exclude)
  static_covariates <- baseline_data[, static_covariate_names, drop = FALSE]

  # Combine static covariates with the grid of all possible histories
  prediction_data <- cbind(static_covariates, history_grid)

  # --- Main Iteration Loop ---
```

```r
  for (t in times) {
    # Identify predictable histories
    predictable_idx <- which(
      !(prediction_data[[col_ypprev]] %in% absorbing_states) &
      !(prediction_data[[col_yprev]] %in% absorbing_states)
    )
    n_predictable <- length(predictable_idx)

    # Subset the data for prediction and set the current time
    prediction_data_subset <- prediction_data[predictable_idx, ]
    prediction_data_subset[[col_time]] <- t

    pred_array_3d <- NULL

    if (detected_type == "brms") {
      # refactor previous states to remove absorbing states
      prediction_data_subset[[col_yprev]] <-
factor(prediction_data_subset[[col_yprev]], levels = c(1:M)[-absorbing_states])
      prediction_data_subset[[col_ypprev]] <-
factor(prediction_data_subset[[col_ypprev]], levels = c(1:M)[-absorbing_states])
      re_arg <- if (include_re) NULL else NA
      brms_pred_long <- marginaleffects::predictions(model_fit,
                                                     newdata = prediction_data_subset,
                                                     type = "response",
                                                     re_formula = re_arg) |>
                        marginaleffects::posterior_draws()

      pred_array_3d <- array(0, dim = c(n_draws, n_predictable, M))
      indices <- cbind(brms_pred_long$drawid,
                       brms_pred_long$rowid,
                       as.numeric(brms_pred_long$group))
      pred_array_3d[indices] <- brms_pred_long$draw

    } else { # detected_type == "rmsb"
    # dimension 1 = 4000 draws
    # dimension 2 = X_{t-1} and X_{t-2} combinations (rows from prediction_data_subset)
    # dimension 3 = cumulative probabilities (e.g., 4 columns for 5 states)
      pred_array_3d <- predict_blrm_manual(model_fit, prediction_data_subset,
include_re = include_re)
    }

    state_probs_all_draws <- do.call(rbind, lapply(1:dim(pred_array_3d)[1], function(d)
pred_array_3d[d, , ]))

    # --- Assemble transition probabilities for each draw ---
    # Create a 4D array to hold all transition probabilities: P(X_t=l | X_{t-2}=h,
X_{t-1}=j)
    # Dims: (draw, h, j, l)
    # So [, 1, 1, 1] give all draws for P(l = 1 | h = 1, j = 1)
    # [, 1, 1, ] for all draws for P(l | h = 1, j = 1) (rows must sum to 1)
    cond_probs_4d_array <- array(0, dim = c(n_draws, M, M, M))

    for (d in 1:n_draws) {
      # Create a placeholder matrix for this specific draw
      # each row is combination of (h, j), each column is l (rows must sum to 1)
```

```
      # this is for one draw only
      cond_probs_matrix_d <- matrix(0, nrow = M * M, ncol = M)

      # Handle absorbing states (P(j->j) = 1 if j is absorbing)
      absorbing_idx <- which(prediction_data[[col_yprev]] %in% absorbing_states)
      if (length(absorbing_idx) > 0) {
        absorbing_state <- as.numeric(as.character(prediction_data[[col_yprev]]
[absorbing_idx]))
        rows_and_cols <- cbind(absorbing_idx, absorbing_state)
        cond_probs_matrix_d[rows_and_cols] <- 1.0
      }

      # Get state probabilities for the current draw
      start_row <- (d - 1) * n_predictable + 1
      end_row <- d * n_predictable
      draw_d_probs <- state_probs_all_draws[start_row:end_row, ]

      # Place them into the correct rows
      cond_probs_matrix_d[predictable_idx, ] <- draw_d_probs

      # Reshape and store in the final 4D array
      cond_probs_4d_array[d, , , ] <- array(cond_probs_matrix_d, dim = c(M, M, M))
    }

    # --- Update Probabilities ---
    # Get joint probabilities from the previous step for all draws: P(X_{t-2}, X_{t-1})
    prev_P_joint_all_draws <- P_joint_bayes[, , , t]
    current_P_joint_all_draws <- array(0, dim = c(n_draws, M, M))

    for (d in 1:n_draws) {
      # This is P(X_{t-2}=h, X_{t-1}=j) for draw d
      prev_P_joint_d <- prev_P_joint_all_draws[d, , ]
      # P(X_t = l | X_{t-1} = j, X_{t-2} = h) for this draw
      cond_probs_d <- cond_probs_4d_array[d, , , ]

      # Law of Total Probability: P(X_t=l|h,j) * P(h,j)
      # weighted_probs <- cond_probs_d * array(prev_P_joint_d, dim = dim(cond_probs_d))
      weighted_probs <- sweep(cond_probs_d, c(1, 2), prev_P_joint_d, "*")
      # Marginalize over the t-2 state (h) to get P(X_{t-1}, X_t)
      # = Summation over h (the state at t-2), resulting in P(X_{t-1}=j, X_t=l)
      current_P_joint_d <- apply(weighted_probs, c(2, 3), sum)

      current_P_joint_all_draws[d, , ] <- current_P_joint_d

      # Marginalize over the t-1 state (j) to get P(X_t)
      uncond_probs_t <- colSums(current_P_joint_d)
      # Consider normalizing  to prevent floating point error accumulation (not doing
this for now)
      P_uncond_bayes[d, , t + 1] <- uncond_probs_t
      # P_uncond_bayes[d, , t + 1] <- uncond_probs_t / sum(uncond_probs_t)
    }

    P_joint_bayes[, , , t + 1] <- current_P_joint_all_draws
  }
```

```
  return(P_uncond_bayes)
}
```

Marginalize over whole population

```
write_SOP <- function(i, tx, baseline_df, path, model){

  row <- baseline_df[i, ]

  sops <-
  sop_markov_second_order(
    model_fit = model,
    baseline_data = data.frame(id = row$id, tx = tx, ecog_fstcnt = row$ecog_fstcnt,
diagnosis = row$diagnosis, albumin = row$albumin,  c_reactive_protein =
row$c_reactive_protein, yprev = row$yprev, ypprev = row$ypprev),
    col_yprev = "yprev",
    col_ypprev = "ypprev",
    col_time = "week",
    times = 1:26,
    n_states = 5,
    absorbing_states = c(5),
    include_re = TRUE
  )

  sops <- reshape2::melt(sops,
                  varnames = c("draw", "state", "time"),
                  value.name = "probability")
  sops <- sops |>
    mutate(
        time  = as.integer(sub("t=", "", time)),
        draw  = as.integer(sub("draw_", "", draw)),
        state = as.factor(sub("State_", "", state)),
        tx    = tx,
        i     = i ) |>
    filter(time > 0)

  folder <- file.path(path, glue::glue("/marginalized_sop_{tx}"), glue::glue("/
msop_{i}.parquet"))

  # Create the directory if it doesn't exist
  dir_to_create <- dirname(folder)
  if (!dir.exists(dir_to_create)) {
    dir.create(dir_to_create, recursive = TRUE)
  }

  arrow::write_parquet(
    x = sops,
    sink = folder
    )
}
```

Marginalized SOPs

Generate SOPs

```
generate_SOPS <- FALSE
```

```
path <- here("2-hosp", "output", "primary-estimand")
future::plan(future::multisession, workers=6)

baseline_df <- df |>
    filter(week == 1) |>
    select(id, week, ecog_fstcnt, diagnosis, albumin, c_reactive_protein, yprev,
ypprev) |>
    distinct() |>
    arrange(id)

# Control
if (generate_SOPS) {
    furrr::future_walk(1:nrow(baseline_df), \(x) write_SOP(x, 0, baseline_df, path,
model = model),
                       .progress=TRUE,
                       .options = furrr_options(seed=TRUE))
}
# Treatment
if (generate_SOPS) {
    furrr::future_walk(1:nrow(baseline_df), \(x) write_SOP(x, 1, baseline_df, path,
model = model),
                       .progress=TRUE,
                       .options = furrr_options(seed=TRUE))
}
```

Read marginalized SOPs

```
files <- fs::dir_ls(here("2-hosp", "output", "primary-estimand", "marginalized_sop_0"),
glob = "*.parquet")

soc_df <- map(files, \(x) arrow::read_parquet(x), .progress=TRUE) |> rbindlist()

# Average SOPs over covariate settings for each state, day, and MCMC draw
soc_df <- soc_df |>
  group_by(state, time, draw, tx) |>
  summarise(sop = mean(probability), .groups = "drop") |>
  mutate(stat = as.factor(state))

files <- fs::dir_ls(here("2-hosp", "output", "primary-estimand", "marginalized_sop_1"),
glob = "*.parquet")
sat_df <- map(files, \(x) arrow::read_parquet(x), .progress=TRUE) |> rbindlist()
sat_df <- sat_df |>
  group_by(state, time, draw, tx) |>
  summarise(sop = mean(probability), .groups = "drop") |>
  mutate(stat = as.factor(state))

marg_df <- rbind(soc_df, sat_df)
```

Visualize Results

```
plot_model <- marg_df |>
  mutate(tx = factor(tx)) |>
  ggplot() +
  aes(x = time, y = sop) +
  ggdist::stat_lineribbon(aes(fill = state, linetype = tx),
                          linewidth = 0.6,
                          alpha = 0.5,
                          .width = c(0.95)) +
  scale_color_brewer(palette = "Dark2", guide="none") +
  scale_fill_brewer(palette = "Dark2") +
  scale_x_continuous(breaks = seq(1, 26, by=4)) +
  scale_y_continuous(breaks = seq(0, 1, by=0.1)) +
  coord_cartesian(ylim = c(0, 1)) +
  labs(x = "Study Week",
   y = "SOP",
   subtitle = "Marginalized state occupancy probabilities",
   fill = "State",
   linetype = "Treatment")
```

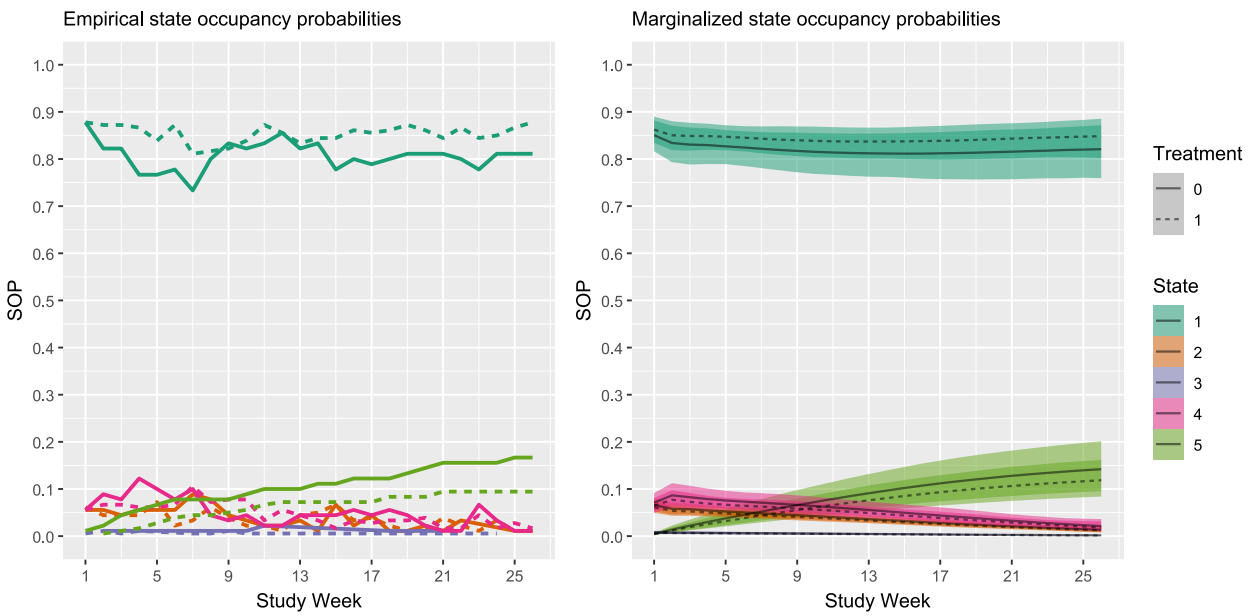
Empirical SOPs

```
df_forward <- read_parquet(here("2-hosp", "sim-data", "ha_08_forward.parquet")) |>
  filter(pat_id %in% c(tx_id, soc_id))

empirical_sop <- df_forward |>
  group_by(tx, week) |>
  count(y) |>
  mutate(
    sop = n / sum(n),
    state = as.factor(y),
    tx = factor(tx)
  ) |>
  ungroup()

plot_empirical <- empirical_sop |>
  ggplot() +
  aes(x = week, y = sop) +
  geom_line(aes(color = state, linetype = tx), linewidth = 1) +
  scale_color_brewer(palette = "Dark2") +
  scale_x_continuous(breaks = seq(1, 26, by=4)) +
  scale_y_continuous(breaks = seq(0, 1, by=0.1)) +
  coord_cartesian(ylim = c(0, 1)) +
  labs(x = "Study Week",
       y = "SOP",
       subtitle = "Empirical state occupancy probabilities",
       color = "State",
       linetype = "Treatment") +
  theme(legend.position = "")
```

We can se that some empirical lines cross, but we KNOW that the treatment effect is constant and proportional, so this is just noise.

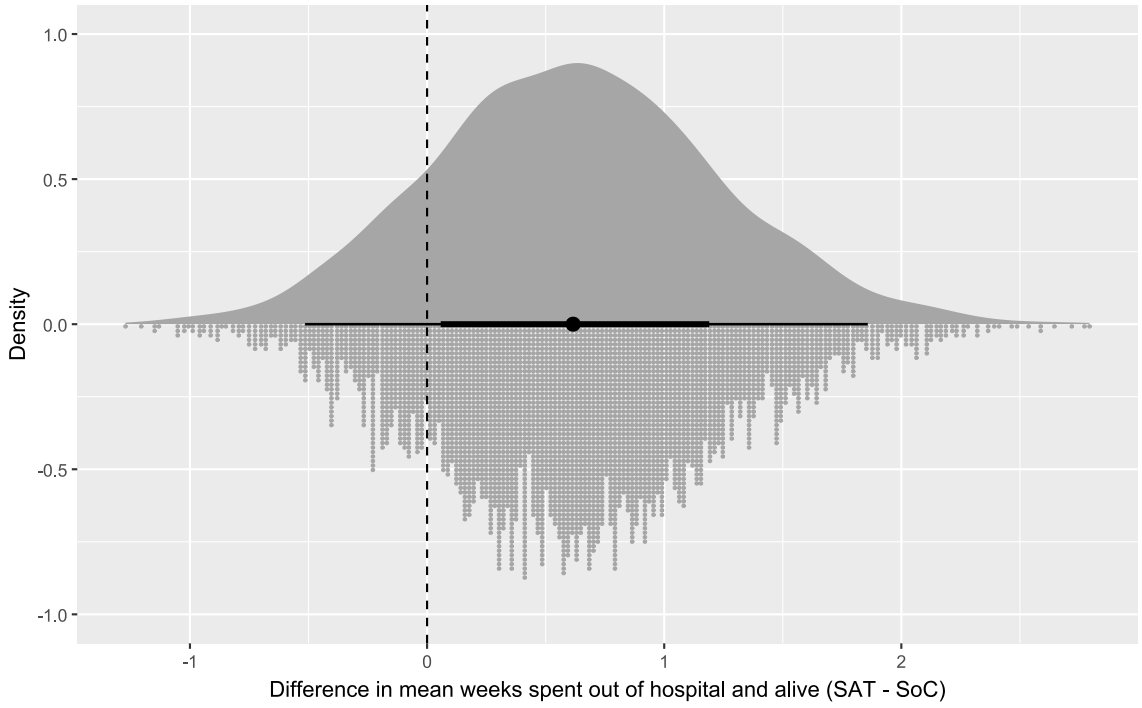


Primary Estimand

Difference is in the mean time spent out of hospital and alive (y = 1).

```
estimand_df <- marg_df |>
  filter(state == "1") |>
  group_by(tx, draw) |>
  summarise(total_time = sum(sop), .groups = "drop") |>
  pivot_wider(names_from = tx, values_from = total_time, names_prefix = "tx_") |>
  mutate(diff = tx_1 - tx_0)

# plot
estimand_df |>
  ggplot() +
  aes(x = diff) +
  ggdist::stat_dots(side = "bottom") +
  ggdist::stat_halfeye() +
  geom_vline(xintercept = 0, linetype = "dashed") +
  labs(x = "Difference in mean weeks spent out of hospital and alive (SAT - SoC)",
       y = "Density")
```




```
ggsave(filename = here("img", "taooh_primary_estimand_conservative-prior.png"), width =
8, height = 5)
```

Checks

Comparing calculated SOPs to brute-force simulation. Note: the optimized simulation was coded by Gemini, but matches a slow version I wrote before. Confidence: high.

When we compare the simulations against the calculations the SOPs match exactly.

```
t1 <- Sys.time()
N <- 4e5
set.seed(1234) # for reproducibility

# --- 1. Pre-calculation of Transition Probabilities ---

# Define simulation parameters
n_states <- 5
absorbing_states <- 5
non_absorbing_states <- 1:(n_states - 1)
times <- 1:26

# Create a grid of every possible non-absorbing history and time point
prediction_grid <- expand.grid(
  ypprev = factor(non_absorbing_states),
  yprev  = factor(non_absorbing_states),
  week   = times
)

# Extract the static baseline covariates from your sample data frame
static_cov_names <- setdiff(names(df), c("ypprev", "yprev", "week"))
baseline_covariates <- df[1, static_cov_names, with = FALSE]
```

```r
# Create the full dataset for the one-time prediction
full_prediction_data <- cbind(baseline_covariates, prediction_grid)

# --- Make one single, large prediction call ---
# This returns a 3D array: [draws, rows_in_grid, states]
all_probs_raw <- predict_blrm_manual(
  model,
  full_prediction_data,
  include_re = FALSE
)

num_draws <- dim(all_probs_raw)[1]

# --- Reshape the raw predictions into a 5D lookup array for fast access ---
# Dimensions: [ypprev, yprev, time, draw, state_outcome]
lookup_table <- array(0,
  dim = c(
    length(non_absorbing_states),
    length(non_absorbing_states),
    length(times),
    num_draws,
    n_states
  ),
  dimnames = list(
    ypprev = non_absorbing_states,
    yprev = non_absorbing_states,
    week = times,
    draw = 1:num_draws,
    state = 1:n_states
  )
)

# Populate the lookup table by mapping the flat prediction output
# back to its structured [h, j, t] coordinates.
for (i in 1:nrow(prediction_grid)) {
  h <- as.integer(as.character(prediction_grid$ypprev[i]))
  j <- as.integer(as.character(prediction_grid$yprev[i]))
  t <- prediction_grid$week[i]

  # Assign probabilities for all draws and all outcome states for this h,j,t
  lookup_table[h, j, t, , ] <- all_probs_raw[, i, ]
}

# Clean up large intermediate objects to save memory
rm(all_probs_raw, full_prediction_data, prediction_grid, baseline_covariates)
gc() # Garbage collection


# --- 2. Simulation Initialization ---

# Initialize the main data object for N individuals
data <- as.data.table(df[rep(1, N), ])

# Pre-assign one fixed posterior draw to each individual for the whole simulation
```

```r
trajectory_draw_indices <- sample.int(num_draws, size = N, replace = TRUE)

# This logical vector tracks who has reached an absorbing state
has_reached_absorbing <- (data$yprev %in% absorbing_states)

# Pre-allocate a list to store the results for each time step
y <- vector("list", length(times))


# --- 3. Main Simulation Loop (Optimized) ---

for (i in times) {

  # Identify individuals who are still active (not in an absorbing state)
  active_indices <- which(!has_reached_absorbing)

  # Early exit if everyone has finished
  if (length(active_indices) == 0) {
    last_y <- y[[i - 1]]
    for (j in i:length(times)) { y[[j]] <- last_y }
    break
  }

  # Get the state history for only the active individuals
  active_ypprev <- as.integer(as.character(data$ypprev[active_indices]))
  active_yprev  <- as.integer(as.character(data$yprev[active_indices]))

  # Get the pre-assigned draw for each active individual
  active_draw_indices <- trajectory_draw_indices[active_indices]

  # --- EFFICIENT PROBABILITY LOOKUP (replaces prediction call) ---
  num_active <- length(active_indices)
  selected_probs <- matrix(nrow = num_active, ncol = n_states)

  # Create an index matrix to extract values from the 5D lookup_table
  # Each row corresponds to an active person: [h, j, t, draw]
  index_matrix <- cbind(
    active_ypprev,
    active_yprev,
    i, # current week
    active_draw_indices
  )

  # Extract the probability for each outcome state using vectorized indexing
  for (cat in 1:n_states) {
    # For each category, append it to the index matrix and extract all values at once
    selected_probs[, cat] <- lookup_table[cbind(index_matrix, cat)]
  }

  # Sample new outcomes using the probabilities from the single, fixed draws
  y_new_active <- apply(selected_probs, 1, function(p) {
    # Handle potential floating point errors where sum(p) is not exactly 1
    sample(1:n_states, size = 1, prob = p)
  })
```

```
  # Initialize the result vector for this time step with the previous state
  y_new <- as.integer(as.character(data$yprev))
  # Update the states for the active individuals with their new sampled state
  y_new[active_indices] <- y_new_active

  # Update the tracker for the NEXT loop iteration
  has_reached_absorbing[active_indices[y_new_active %in% absorbing_states]] <- TRUE

  # Update the history variables in the main data object for the next iteration
  data$ypprev <- data$yprev
  data$yprev <- y_new

  # Store the full vector of results for this time step
  y[[i]] <- y_new
  message("Iteration ", i, " complete. Active individuals: ", num_active)
}


# --- 4. Post-Processing and Results ---

# Function to calculate state proportions for a given time step
calculate_proportions_matrix <- function(vec, all_levels) {
  # Ensure all possible states are represented in the table, even if count is 0
  counts <- table(factor(vec, levels = all_levels))
  proportions <- prop.table(counts)
  return(proportions)
}

# Apply the function to the list of results
all_possible_states <- 1:n_states
proportions_list <- lapply(y, calculate_proportions_matrix, all_levels =
all_possible_states)

# Bind the results into a final, easy-to-read matrix
proportion_matrix <- do.call(rbind, proportions_list)
rownames(proportion_matrix) <- paste0("Week_", times)
t2 <- Sys.time()
t2 - t1
```

```
t1 <- Sys.time()
sops_2 <- sop_markov_second_order(
  model_fit = model,
  baseline_data = as.data.frame(df[1, ]),
  col_yprev = "yprev",
  col_ypprev = "ypprev",
  col_time = "week",
  times = 1:26,
  n_states = 5,
  absorbing_states = c(5),
  include_re = FALSE
)
t2 <- Sys.time()
t2 - t1
```

```
Time difference of 15.40865 secs
```

```
# Differences in Calculated and Simulated SOPs
format(
  round(t(apply(sops_2, c(2,3), mean))[-1,], digits = 4) - round(proportion_matrix,
digits = 4),
  scientific = FALSE
)
```

```
     State_1   State_2   State_3   State_4   State_5
t=1  " 0.0003" " 0.0001" " 0.0000" "-0.0002" "-0.0001"
t=2  "-0.0004" " 0.0005" "-0.0001" "-0.0001" "-0.0001"
t=3  " 0.0002" " 0.0001" " 0.0000" "-0.0001" "-0.0002"
t=4  " 0.0001" " 0.0002" " 0.0001" "-0.0001" "-0.0004"
t=5  "-0.0001" " 0.0003" " 0.0001" " 0.0000" "-0.0003"
t=6  " 0.0005" "-0.0001" " 0.0001" "-0.0003" "-0.0002"
t=7  " 0.0007" "-0.0003" " 0.0000" "-0.0002" "-0.0002"
t=8  " 0.0007" "-0.0001" " 0.0000" "-0.0004" "-0.0003"
t=9  " 0.0001" " 0.0000" "-0.0001" " 0.0002" "-0.0003"
t=10 " 0.0000" " 0.0000" " 0.0000" " 0.0003" "-0.0004"
t=11 " 0.0009" "-0.0001" "-0.0001" "-0.0003" "-0.0004"
t=12 " 0.0009" "-0.0001" " 0.0000" "-0.0004" "-0.0004"
t=13 " 0.0003" "-0.0002" " 0.0000" " 0.0002" "-0.0003"
t=14 " 0.0004" "-0.0002" " 0.0000" " 0.0001" "-0.0005"
t=15 " 0.0004" " 0.0000" " 0.0000" " 0.0001" "-0.0004"
t=16 " 0.0004" "-0.0004" "-0.0001" " 0.0003" "-0.0003"
t=17 " 0.0004" "-0.0001" " 0.0000" "-0.0001" "-0.0003"
t=18 " 0.0000" " 0.0003" "-0.0001" " 0.0000" "-0.0003"
t=19 "-0.0001" " 0.0002" "-0.0001" " 0.0002" "-0.0003"
t=20 " 0.0000" " 0.0002" " 0.0001" " 0.0000" "-0.0003"
t=21 " 0.0000" " 0.0002" " 0.0000" " 0.0000" "-0.0002"
t=22 " 0.0002" "-0.0001" " 0.0000" " 0.0002" "-0.0002"
t=23 " 0.0003" "-0.0002" " 0.0000" " 0.0001" "-0.0002"
t=24 " 0.0004" " 0.0000" " 0.0000" "-0.0002" "-0.0002"
t=25 " 0.0003" " 0.0000" "-0.0001" " 0.0000" "-0.0002"
t=26 " 0.0005" "-0.0001" "-0.0001" " 0.0001" "-0.0003"
```